

GROUP ASSIGNMENT – 1

Problem Statement 1:

A rescue robot is operating inside a rectangular grid representing a building. The robot must move from a starting position to a target position while avoiding obstacles.

The grid contains four types of cells:

S – Starting position

G – Goal position

. – Free cell

– Obstacle

The robot can move Up, Down, Left, or Right. Each movement has a cost of 1.

Objective

Determine a path from S to G using:

Breadth First Search (BFS)

Depth First Search (DFS)

Input Format

First Line

R C

Rows and columns of the grid.

Next R Lines

A grid containing S, G, ., and #.

Output Format

For each algorithm, print:

Algorithm: BFS

Path Found: Yes

Path: RIGHT RIGHT DOWN DOWN ...

Number of Moves = K

Nodes Expanded = N

Similarly for DFS.

If no path exists, print:

Path Found: No

Constraints

- $1 \leq R, C \leq 100$
 - Exactly one S and one G exist
 - The robot cannot move through #
 - The robot cannot move outside the grid
 - Each movement has a cost of 1
 - Successors must be generated in the order: UP → RIGHT → DOWN → LEFT
-

Sample Input 1

```
5 5
S..#.
#..#.
.....
.##..
....G
```

Sample Output 1

Algorithm: BFS

Path Found: Yes

Path: RIGHT RIGHT DOWN DOWN RIGHT RIGHT DOWN DOWN

Number of Moves = 8

Nodes Expanded = N

Algorithm: DFS

Path Found: Yes

Path: RIGHT RIGHT DOWN DOWN RIGHT RIGHT DOWN DOWN

Number of Moves = 8

Nodes Expanded = N

The exact number of nodes expanded may depend on the implementation.

Sample Input 2

```
5 5
S....
####.
...#
.####
....G
```

Sample Output 2

Algorithm: BFS

Path Found: No

Algorithm: DFS

Path Found: No

State Representation

A state is represented as:

(row, column)

where (row, column) denotes the current position of the robot.

Example:

(0, 0)

Successor Function

From any state (r, c), the following actions are possible:

- UP $\rightarrow (r-1, c)$
- RIGHT $\rightarrow (r, c+1)$
- DOWN $\rightarrow (r+1, c)$
- LEFT $\rightarrow (r, c-1)$

A successor is valid only if:

- It is inside the grid
- It is not an obstacle
- It has not already been visited

Additional Requirements

For BFS and DFS report:

- Path Found
- Number of Moves
- Nodes Expanded
- Execution Time

Comparison

Compare BFS and DFS based on:

- Path length
 - Number of states explored
 - Execution time
 - Optimality of the solution
-

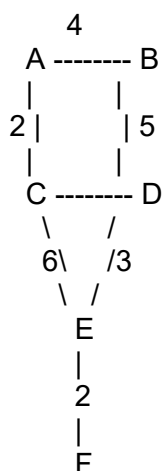
Problem Statement 2:

A university campus consists of several locations connected by roads. Each road has a travel cost. A heuristic value representing the estimated cost from each location to the destination is also provided.

The objective is to find a route from a given starting location to a target location using informed search.

Campus Graph

Use the following graph:



The edges are bidirectional.

Edge Costs

Edge	Cost
------	------

A – B	4
-------	---

A – C	2
-------	---

B – D	5
-------	---

C – D	3
-------	---

C – E	6
-------	---

D – E	3
-------	---

E – F	2
-------	---

Heuristic Values

The goal is F.

Node	$h(n)$
------	--------

A	7
---	---

B	8
---	---

C	5
---	---

D	4
---	---

E	2
---	---

F	0
---	---

Objective

Find a path from A to F using:

Greedy Best-First Search

A* Search

Input Format

First Line

N M

Next M Lines

U V C

where U and V are connected nodes and C is the edge cost.

Next Line

S G

representing start and goal.

Next N Lines

Node Heuristic

Output Format

For each algorithm:

Algorithm: Greedy Best-First Search

Path Found: Yes

Path: $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Total Cost = K

Nodes Expanded = N

Similarly for A*.

Search Functions

For Greedy Best-First Search:

$$f(n) = h(n)$$

For A*:

$$f(n) = g(n) + h(n)$$

where:

$g(n)$ = cost from start to current node

$h(n)$ = estimated cost to goal

constraints

- $2 \leq N \leq 100$
- $1 \leq M \leq N(N-1)/2$
- Edge costs are positive
- $h(n) \geq 0$
- Exactly one start and goal node
- Graph edges are bidirectional

Sample Test Case 1

6 7
A B 4
A C 2
B D 5
C D 3
C E 6
D E 3
E F 2
A F
A 7
B 8
C 5
D 4
E 2
F 0

Sample Output

Algorithm: Greedy Best-First Search
Path Found: Yes
Path: $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$
Total Cost = 10
Nodes Expanded = N

Algorithm: A* Search
Path Found: Yes
Path: $A \rightarrow C \rightarrow D \rightarrow E \rightarrow F$
Total Cost = 10
Nodes Expanded = N

Sample Test Case 2

Provide a second graph in which Greedy Best-First Search and A* are likely to choose different routes, allowing students to observe that Greedy Search does not necessarily produce the optimal path.

Additional Requirements

For both algorithms report:

- Path Found
- Path
- Total Path Cost
- Nodes Expanded
- Execution Time

Comparison

Compare Greedy Best-First Search and A* based on:

- Path cost
 - Number of states explored
 - Execution time
 - Optimality
 - Effectiveness of the heuristic used
-

Problem Statement 3:

Robot Strategy Battle

Two autonomous robots compete to collect energy cells on a warehouse grid.

MAX robot = A

MIN robot = B

Energy cell = E

Obstacle = #

Empty cell = .

Robots take alternate turns. On each turn, a robot can move one cell Up, Right, Down, or Left.

When a robot reaches an energy cell, it collects it and receives points.

The objective is for MAX to maximize its final score while MIN attempts to minimize MAX's advantage.

Initial Board

```
+---+---+---+---+---+---+
| A | . | E | . | # | . | . |
+---+---+---+---+---+---+
| . | # | . | . | # | E | . |
+---+---+---+---+---+---+
| . | . | . | # | . | . | . |
+---+---+---+---+---+---+
| # | E | # | . | . | # | . |
+---+---+---+---+---+---+
| . | . | . | . | E | . | B |
+---+---+---+---+---+---+
```

Each E is worth 10 points.

Problem Description

Minimax

Minimax with Alpha-Beta Pruning

The AI must determine the best move for MAX using a depth-limited game tree.

Input Format

First Line

R C

Next R Lines

The game board.

Next Line

MAX

Next Line

D

Search depth.

Sample Input

```
5 7
A.E.#..
#..#E.
...#...
#E#..#.
....E.B
MAX
5
```

Output Format

For Minimax:

Algorithm: Minimax
Best Move: RIGHT
Evaluation Score = 20
Nodes Generated = N
Nodes Expanded = N
Execution Time = T

For Alpha-Beta:

Algorithm: Alpha-Beta
Best Move: RIGHT
Evaluation Score = 20
Nodes Generated = N
Nodes Expanded = N
Nodes Pruned = P
Execution Time = T
State Representation

A state contains:

(A_position,
B_position,
remaining_energy_cells,
current_player,
score)
Successor Function

For every valid movement:

UP
RIGHT
DOWN
LEFT

generate a new game state.

If a robot reaches an uncollected E:

score += 10

and that energy cell is removed from the state.

Evaluation Function

At non-terminal states:

Evaluation =
MAX Score - MIN Score
+ positional advantage

A higher value is better for MAX.

Alpha-Beta Pruning

Use:

α = best value guaranteed for MAX

β = best value guaranteed for MIN

Prune a branch when:

$\alpha \geq \beta$

Move Ordering

For Alpha-Beta, test two cases:

Case 1:

Normal move order:

UP $\rightarrow$ RIGHT $\rightarrow$ DOWN $\rightarrow$ LEFT

Case 2:

Heuristic move order:

Moves collecting an energy cell first

Constraints

- $3 \leq R, C \leq 15$
- Exactly one A and one B
- At least one energy cell exists
- Each energy cell is worth 10 points
- Search depth $1 \leq D \leq 8$
- Robots cannot enter #
- A collected energy cell cannot be collected again

Sample Test Case 2

5 7

A...E..

.###...

..E....

...###.

.E...B.

MAX

4

Additional Requirements

For both algorithms report:

- Best Move
- Evaluation Score
- Nodes Generated
- Nodes Expanded
- Search Depth
- Execution Time

For Alpha-Beta additionally report:

- Nodes Pruned

Comparison

Compare:

Minimax vs Alpha-Beta

based on:

- Nodes expanded
- Nodes pruned
- Execution time
- Best move
- Evaluation score

Also compare:

Alpha-Beta without move ordering vs Alpha-Beta with move ordering.

Problem Statement – 04

University Exam Timetable Optimization Using Local Search

1. Problem Statement

A university must prepare an examination timetable for multiple courses.

Each course examination must be assigned to one of the available examination time slots while satisfying scheduling constraints. Some pairs of courses have common students and therefore should ideally not be scheduled at the same time.

The university has a limited number of examination slots. The objective is to create an examination timetable that:

1. Minimizes conflicts between courses that have common students.
2. Minimizes the total number of conflicting course pairs scheduled in the same examination slot.
3. Avoids placing too many courses in the same examination slot.
4. Reduces undesirable examination scheduling conflicts.

The timetable must be optimized using a Local Search algorithm based on Hill Climbing.

Objective

Find an examination timetable with minimum total conflict penalty and minimum distribution penalty.

2. Group Task

Each group must design and implement an Exam Timetable Optimization System.

The system must:

- Represent an examination timetable as a state.
- Calculate the quality of a timetable using a cost function.
- Generate neighboring timetable states.
- Apply the Hill Climbing algorithm to improve the timetable.
- Stop when no better neighboring timetable exists.
- Compare the results obtained using different initial timetables.

The main objective is:

Find a timetable with minimum conflict and minimum penalty.

3. Courses and Examination Slots

Assume the university has the following six courses:

- C1 – Artificial Intelligence
- C2 – Machine Learning
- C3 – Data Structures
- C4 – Database Systems
- C5 – Computer Networks
- C6 – Operating Systems

The university provides four examination slots:

- Slot 1 – Day 1 Morning
- Slot 2 – Day 1 Afternoon
- Slot 3 – Day 2 Morning
- Slot 4 – Day 2 Afternoon

Each course must be assigned to exactly one examination slot.

For this problem:

$N = 6$

$S = 4$

Where:

N = Number of Courses

S = Number of Examination Slots

4. Course Conflicts

Some pairs of courses have common students.

The following course pairs should ideally not be scheduled in the same examination slot:

- C1 – C2
- C1 – C3
- C2 – C4
- C2 – C5
- C3 – C4
- C3 – C6
- C4 – C5
- C5 – C6

For example:

C1 and C2 have common students.

If C1 and C2 are assigned to the same examination slot, a conflict occurs and a penalty is added to the timetable cost.

The conflicts are treated as soft constraints.

Therefore, conflicting courses may appear in the same slot, but doing so increases the total timetable cost.

5. State Representation

A state represents the examination slot assigned to every course.

For example:

Courses: [C1, C2, C3, C4, C5, C6]

State: [1, 2, 3, 1, 4, 2]

This means:

C1 → Slot 1

C2 → Slot 2

C3 → Slot 3

C4 → Slot 1

C5 → Slot 4

C6 → Slot 2

The same timetable can also be represented as:

Slot 1: C1, C4

Slot 2: C2, C6

Slot 3: C3

Slot 4: C5

The index of the array represents the course.

The value represents the examination slot assigned to that course.

6. Initial State

The program receives an initial examination timetable.

Example:

1 1 2 3 4 2

This represents:

C1 → Slot 1

C2 → Slot 1

C3 → Slot 2

C4 → Slot 3

C5 → Slot 4

C6 → Slot 2

In this initial timetable:

- C1 and C2 are both assigned to Slot 1.
- C3 and C6 are both assigned to Slot 2.

Both pairs are conflicting course pairs.

Therefore, the initial timetable contains two conflicts.

Hill Climbing must attempt to improve the timetable by exploring neighboring timetable states.

7. Objective Function

The objective is to minimize the total timetable penalty.

The cost function contains two components:

Total Cost = Conflict Penalty + Distribution Penalty

7.1 Conflict Penalty

For every pair of conflicting courses scheduled in the same examination slot:

Penalty = 10

Therefore:

Conflict Penalty = Number of Conflicts × 10

Example:

If C1 and C2 are assigned to the same slot:

Conflict Penalty = 1×10
= 10

If three conflicting pairs occur in the timetable:

Conflict Penalty = 3×10
= 30

7.2 Distribution Penalty

The timetable should avoid placing too many courses in the same examination slot.

A maximum of two courses in a slot is allowed without any distribution penalty.

For every course beyond two courses in a single slot:

Penalty = 2

For each examination slot:

Excess Courses = $\max(0, \text{Number of Courses in the Slot} - 2)$

The total number of excess courses is the sum of excess courses across all examination slots.

Example:

Slot 1 contains 4 courses.

Two courses are allowed without penalty.

The remaining two courses are excess courses.

$$\begin{aligned}\text{Excess Courses} &= 4 - 2 \\ &= 2\end{aligned}$$

Therefore:

$$\begin{aligned}\text{Distribution Penalty} &= \text{Excess Courses} \times 2 \\ &= 2 \times 2 \\ &= 4\end{aligned}$$

Final Cost Function

$$\begin{aligned}\text{Total Cost} &= 10 \times \text{Number of Course Conflicts} \\ &\quad + 2 \times \text{Total Number of Excess Courses}\end{aligned}$$

The objective is:

Minimize Total Cost

A lower cost represents a better timetable.

The ideal solution has:

$$\text{Total Cost} = 0$$

8. Neighborhood Function

A neighboring solution is generated by changing the examination slot of exactly one course.

For example, consider the current state:

[1, 1, 2, 3, 4, 2]

Move C2 from Slot 1 to Slot 3:

[1, 3, 2, 3, 4, 2]

This produces one neighboring timetable.

Another possible neighbor is obtained by moving C5 from Slot 4 to Slot 2:

[1, 1, 2, 3, 2, 2]

For every course, the algorithm must consider assigning that course to every other possible examination slot.

Therefore:

$$\text{Number of Neighbors} = N \times (S - 1)$$

For this problem:

$$\begin{aligned}\text{Number of Neighbors} &= 6 \times (4 - 1) \\ &= 6 \times 3 \\ &= 18\end{aligned}$$

Therefore, each timetable state has 18 possible neighboring states.

All valid neighboring states should be considered.

9. Hill Climbing Algorithm

The Hill Climbing algorithm works as follows:

Initial Timetable



Calculate Current Cost



Generate All Neighboring Timetables



Calculate Cost of Every Neighbor



Find the Best Neighbor

↓
Is the Best Neighbor Better Than Current State?

 / \
 Yes No
 ↓ ↓
Move to Stop
Neighbor

↓
Repeat

The algorithm performs the following steps:

1. Start with the given initial timetable.
2. Calculate the cost of the current timetable.
3. Generate all neighboring timetable states.
4. Calculate the cost of every neighboring state.
5. Select the neighbor with the lowest cost.
6. If multiple neighboring states have the same lowest cost, select the first such neighbor generated.
7. Move to the selected neighbor only if its cost is strictly lower than the current cost.
8. Repeat the process.
9. Stop when no neighboring state has a lower cost.

The termination condition is called:

Local Optimum

A local optimum does not necessarily mean that the globally best timetable has been found.

10. Input Format

The input format is:

N S

K

Course1 Course2

Course1 Course2

...

Initial Timetable

Where:

N = Number of Courses

S = Number of Examination Slots

K = Number of Conflicting Course Pairs

Each of the next K lines contains two course numbers representing a conflicting pair.

The final line contains the initial timetable.

11. Sample Input

6 4

8

1 2

1 3

2 4

2 5

3 4

3 6

4 5

5 6

1 1 2 3 4 2

This means:

Number of Courses = 6

Number of Slots = 4

Number of Conflict Pairs = 8

Initial timetable:

C1 → Slot 1

C2 → Slot 1

C3 → Slot 2

C4 → Slot 3

C5 → Slot 4

C6 → Slot 2

12. Sample Initial Cost Calculation

Initial state:

[1, 1, 2, 3, 4, 2]

Conflicting pairs:

C1–C2 → Same Slot 1 → Conflict

C1–C3 → Different Slots

C2–C4 → Different Slots

C2–C5 → Different Slots

C3–C4 → Different Slots

C3–C6 → Same Slot 2 → Conflict

C4–C5 → Different Slots

C5–C6 → Different Slots

Therefore:

Number of Conflicts = 2

Conflict Penalty = Number of Conflicts × 10

= 2 × 10

= 20

Slot distribution:

Slot 1 → 2 courses

Slot 2 → 2 courses

Slot 3 → 1 course

Slot 4 → 1 course

There are no courses beyond two in any slot.

Therefore:

Distribution Penalty = 0

Final Initial Cost:

Total Cost = Conflict Penalty + Distribution Penalty

= 20 + 0

= 20

13. Required Implementation

Each group must implement the following modules.

Module 1: Input Processing

Read:

- Number of courses.
- Number of examination slots.
- Number of conflicting course pairs.
- Conflicting course pairs.
- Initial timetable.

Store the conflicts efficiently.

For example, conflicts may be stored using:

- A list of pairs, or
- An adjacency list, or
- An adjacency matrix.

Module 2: State Representation

Represent the examination timetable using a list or array.

Example:

[1, 1, 2, 3, 4, 2]

The index represents the course.

The value represents the examination slot assigned to that course.

Module 3: Cost Function

Implement:

`calculate_cost(state)`

The function must calculate:

1. Number of course conflicts.
2. Conflict penalty.
3. Number of excess courses in all slots.
4. Distribution penalty.
5. Total timetable cost.

The function may return the values required for displaying detailed results.

Module 4: Neighbor Generation

Generate neighboring states by changing the slot of exactly one course.

Example:

Current state:

[1, 1, 2, 3, 4, 2]

Possible neighbors include:

[2, 1, 2, 3, 4, 2]

[3, 1, 2, 3, 4, 2]

[4, 1, 2, 3, 4, 2]

[1, 2, 2, 3, 4, 2]

[1, 3, 2, 3, 4, 2]

[1, 4, 2, 3, 4, 2]

Continue this process for every course.

All possible neighbors obtained by changing exactly one course to a different slot must be considered.

For the sample problem:

$$\begin{aligned}\text{Number of Neighbors} &= 6 \times 3 \\ &= 18\end{aligned}$$

Module 5: Hill Climbing

Implement Hill Climbing using the following steps:

1. Start with the initial timetable.
2. Calculate its cost.

3. Generate all neighboring states.
4. Calculate the cost of every neighbor.
5. Select the best neighbor.
6. If multiple neighbors have the same minimum cost, select the first one generated.
7. Move to the best neighbor only if it has a strictly lower cost than the current timetable.
8. Repeat until no better neighbor exists.
9. Return the final timetable and performance statistics.

14. Required Output

For each experiment, display:

```
=====
Algorithm: Hill Climbing
Problem: Exam Timetable Optimization
=====
```

Initial Timetable:

```
C1 → Slot X
C2 → Slot X
C3 → Slot X
C4 → Slot X
C5 → Slot X
C6 → Slot X
```

```
Initial Conflict Cost = X
Initial Distribution Cost = X
Initial Total Cost = X
```

```
-----
```

Iteration 1

Best Neighbor Timetable:

```
C1 → Slot X
C2 → Slot X
C3 → Slot X
C4 → Slot X
C5 → Slot X
C6 → Slot X
```

```
Conflict Cost = X
Distribution Cost = X
Total Cost = X
```

```
-----
```

Iteration 2

...

```
=====
```

Final Timetable:

C1 → Slot X
C2 → Slot X
C3 → Slot X
C4 → Slot X
C5 → Slot X
C6 → Slot X

Final Conflict Cost = X
Final Distribution Cost = X
Final Cost = X

Iterations = X
Total Neighboring States Evaluated = X
Execution Time = X seconds

Termination Reason: Local Optimum

=====

All values must be calculated dynamically by the program.

The total number of neighboring states evaluated is defined as:

The total number of neighboring states for which the cost function was calculated during the entire Hill Climbing process.

For example, if four iterations of neighbor evaluation are performed and each iteration evaluates 18 neighbors:

Total Neighboring States Evaluated = 4×18
= 72

The final iteration, in which the algorithm finds no better neighbor and stops, must also be included in the count.

15. Experimental Requirement

Each group must run the Hill Climbing algorithm using at least three different initial timetables.

For example:

Experiment 1

[1, 1, 2, 3, 4, 2]

Experiment 2

[2, 3, 1, 1, 4, 4]

Experiment 3

[4, 2, 3, 1, 2, 3]

For every experiment, record:

- Initial timetable.
 - Initial cost.
 - Final timetable.
 - Final cost.
 - Number of iterations.
 - Number of neighboring states evaluated.
 - Execution time.
 - Termination reason.
-

16. Experimental Comparison

After completing all experiments, prepare a comparison table.

Experiment	Initial State	Initial Cost	Final State	Final Cost	Iterations	Neighbors Evaluated	Execution Time
1	[1,1,2,3,4,2]	X	[...]	X	X	X	X
2	[2,3,1,1,4,4]	X	[...]	X	X	X	X
3	[4,2,3,1,2,3]	X	[...]	X	X	X	X

Based on the results, discuss:

1. Which initial timetable produced the best final solution?
2. Did all initial states reach the same final timetable?
3. Did Hill Climbing get stuck in different local optima?
4. How did the initial state affect the number of iterations?
5. How did the initial state affect the final cost?

17. Final Objective

The objective of the project is to implement a Local Search optimization system using Hill Climbing to solve an examination timetable problem.

The objective is to find a timetable with minimum conflict and minimum penalty.

The ideal solution should have:

Conflict Penalty = 0

Distribution Penalty = 0

Therefore:

Ideal Total Cost = 0